

Cross-platform development



Flutter

Asynchronous API (cont.)

- There are certain *widgets* that allow to associate a certain method/request that returns a `Future<T>` object, from which the UI will be updated based on its state
 - For example, `FutureBuilder<T>`
- Alternatively, a request can be made in an `async` method and, when the result is obtained and processing is completed, `setState(() { })` is called, so that the state is updated with the data

Exercise

- Example using the `FutureBuilder<T>` widget
 - Create a fake method for time-consuming data generation

```
Future<String> _fetchAnAsyncString() async {  
  await Future.delayed(const Duration(seconds: 5));  
  return Future.value('Hello world, from an aysnc call!');  
}
```

- Add the widget to the screen

```
FutureBuilder<String>(  
  future: _fetchAnAsyncString(),  
  builder: (  
    BuildContext context, AsyncSnapshot<String> snapshot  
  ) {  
    if (snapshot.hasData) {  
      return Text(snapshot.data!);  
    } else if (snapshot.hasError) {  
      return const Text('Oops, something happened');  
    } else {  
      return const CircularProgressIndicator();  
    }  
  },  
)
```

Official HTTP Library

- To use the REST API, there are several libraries available at <http://pub.dev>
- Depending on the use cases/complexity of what it is intended to implement in the application, libraries that offer better abstraction support for different functionalities can be chosen
 - For example: *interceptors, serializers, cache managers*, etc.

Official HTTP Library

- Dart offers, *out-of-the-box*, a lower-level API that requires the implementation of an `HttpClient` object. However, for simplified REST use, the official `http` library can be used
 - For this, the library `'http: any'` must be included in the `dependencies` section in the `pubspec.yaml` file
 - After that, run the `pub get` command
 - In the Dart file where it is necessary to use the library, do
`import 'package:http/http.dart' as http;`

```
dependencies:  
  flutter:  
    sdk: flutter  
  
# The following adds the Cupertino Icons font to your application.  
# Use with the CupertinoIcons class for iOS style icons.  
cupertino_icons: ^1.0.2  
http: any
```

Exercise

- Access the website <https://catfact.ninja> and check the REST requests that are available to obtain a set of facts about cats
- Using the Dart REST API (`http` library) obtain data related to the *endpoint*: <https://catfact.ninja/facts>
- Get the data in a `FutureBuilder <Response>` and show the data

```
static const String _catFactsUrl = 'https://catfact.ninja/facts';
```

```
FutureBuilder<http.Response>(
  future: http.get(Uri.parse(_catFactsUrl)),
  builder: (
    BuildContext context, AsyncSnapshot<http.Response> snapshot
  ) {
    if (snapshot.hasData) {
      return Expanded(
        child: SingleChildScrollView(
          child: Text(snapshot.data!.body)
        )
      );
    } else if (snapshot.hasError) {
      return const Text('Oops, something happened');
    } else {
      return const CircularProgressIndicator();
    }
  },
)
```

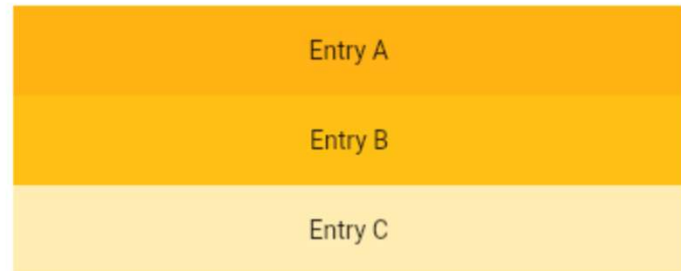
List view

- As shown in the previous example, Flutter offers some widgets for simplified scrollable creation views
 - To make “any” widget scrollable , it can be wrapped in a `SingleChildScrollView`
 - For example, it can be used to scroll longer text or a set of widgets
 - To enable scrolling, it is important to ensure that the definitions of the widgets with which they are associated are not incompatible
 - A scrollable view implies that a widget does not have vertical constraints defined

List view

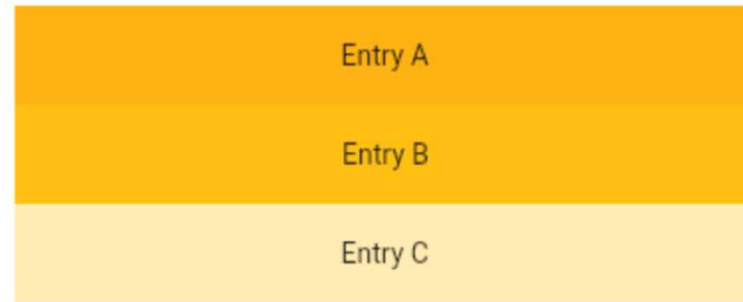
- To implement lists more efficiently, `ListView` can be used
 - Alternatively, a `ListView.builder` can be used, which helps in building it
- There are other constructors for the `ListView` that may be equally useful, for example, `ListView.separated` which offers a constructor that allows the introduction of a separator

ListView



```
ListView(  
  padding: const EdgeInsets.all(8),  
  children: <Widget>[  
    Container(  
      height: 50,  
      color: Colors.amber[600],  
      child: const Center(child: Text('Entry A')),  
    ),  
    Container(  
      height: 50,  
      color: Colors.amber[500],  
      child: const Center(child: Text('Entry B')),  
    ),  
    Container(  
      height: 50,  
      color: Colors.amber[100],  
      child: const Center(child: Text('Entry C')),  
    ),  
  ],  
)
```

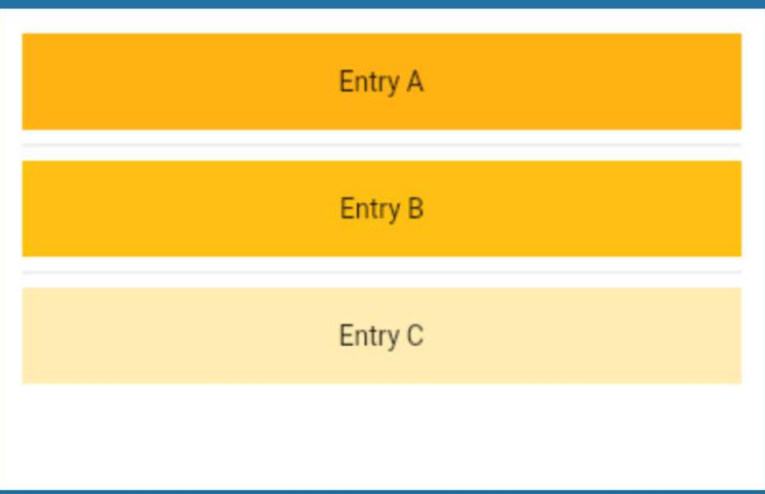
ListView.builder



```
final List<String> entries = <String>['A', 'B', 'C'];
final List<int> colorCodes = <int>[600, 500, 100];

ListView.builder(
  padding: const EdgeInsets.all(8),
  itemCount: entries.length,
  itemBuilder: (BuildContext context, int index) {
    return Container(
      height: 50,
      color: Colors.amber[colorCodes[index]],
      child: Center(child: Text('Entry ${entries[index]}')),
    );
  }
);
```

ListView.separated



```
final List<String> entries = <String>['A', 'B', 'C'];
final List<int> colorCodes = <int>[600, 500, 100];

ListView.separated(
  padding: const EdgeInsets.all(8),
  itemCount: entries.length,
  itemBuilder: (BuildContext context, int index) {
    return Container(
      height: 50,
      color: Colors.amber[colorCodes[index]],
      child: Center(child: Text('Entry ${entries[index]}')),
    );
  },
  separatorBuilder: (BuildContext context, int index) => const Divider(),
);
```

Exercise

- Continue the previous exercise, processing the information received to display it in a `ListView`
- Create a data model for the object corresponding to a fact about *cats*
- Add a property to the screen state that allows to display a loading widget, e.g., `CircularProgressIndicator`, while the request is made

Exercise

- Class to represent a fact

```
class CatFact {  
    CatFact.fromJson(Map<String, dynamic> json)  
        : fact = json['fact'],  
          length = json['length'];  
  
    final String fact;  
    final int length;  
}
```

Exercise

- Function to get the data

```
List<CatFact>? _catFacts;
bool _fetchingData = false;

Future<void> _fetchCatFacts() async {
  try {
    setState(() => _fetchingData = true);

    http.Response response = await http.get(Uri.parse(_catFactsUrl));

    if (response.statusCode == HttpStatus.ok) {
      debugPrint(response.body);
      final Map<String, dynamic> decodedData = json.decode(response.body);
      setState(() => _catFacts = (decodedData['data'] as List)
        .map((fact) => CatFact.fromJson(fact)).toList());
    }
  } catch (ex) {
    debugPrint('Something went wrong: $ex');
  } finally {
    setState(() => _fetchingData = false);
  }
}
```

Exercise

- Create `ListView` with the obtained data or representation of the state of obtaining the data

```
ElevatedButton(  
    onPressed: _fetchCatFacts,  
    child: const Text('Fetch cat facts'),  
) ,  
if (_fetchingData) const CircularProgressIndicator(),  
if (!_fetchingData && _catFacts != null && _catFacts!.isNotEmpty)  
    Expanded(  
        child: ListView.separated(  
            itemCount: _catFacts!.length,  
            separatorBuilder: (_, __) => const Divider(thickness: 2.0),  
            itemBuilder: (BuildContext context, int index) => ListTile(  
                title: Text('Cat fact #$index'),  
                subtitle: Text(_catFacts![index].fact),  
            ),  
        ),  
    ),  
)
```

Image insertion example

- Image distributed with the application

- Create an `images` directory in the project
- Place an image of your choice in the directory

`.../<flutter_prj_path>/images/xpto.jpg`

- Add (at the end of the `pubspec.yaml` file):

```
flutter:
```

```
...
```

```
assets:
```

```
- images/xpto.jpg
```

- Add the image to a screen

```
Image.asset('images/xpto.jpg')
```

- Alternative:

```
SizedBox(height:200, child: Image.asset('images/xpto.jpg'))
```


Image insertion example

- Image available on the internet
 - Add to a screen an image that can be obtained via a *link*

```
static const String _ipc_logo =  
    'https://wayf.ipc.pt/IPCds/images/logo_ipc2.png';  
...  
SizedBox(height:50, child: Image.network(_ipc_logo)),  
...
```

Text editing boxes

- `TextField` or `TextFormField` widgets are typically used to allow the user to enter text
- Example for creating an edit box for the existing increment value in the Flutter base example

```
...
TextField(
  decoration: const InputDecoration(
    labelText: 'Increment:',
    hintText: 'Value to increment',
  ),
  onChanged: (value) => changeInc(int.tryParse(value) ?? 1),
),
...
```

- Assuming the existence of the following code:

```
int _inc=1;
void changeInc(int inc) { setState(() {_inc = inc;}); }
void _incrementCounter() { setState(() {_counter+=_inc; }); }
```

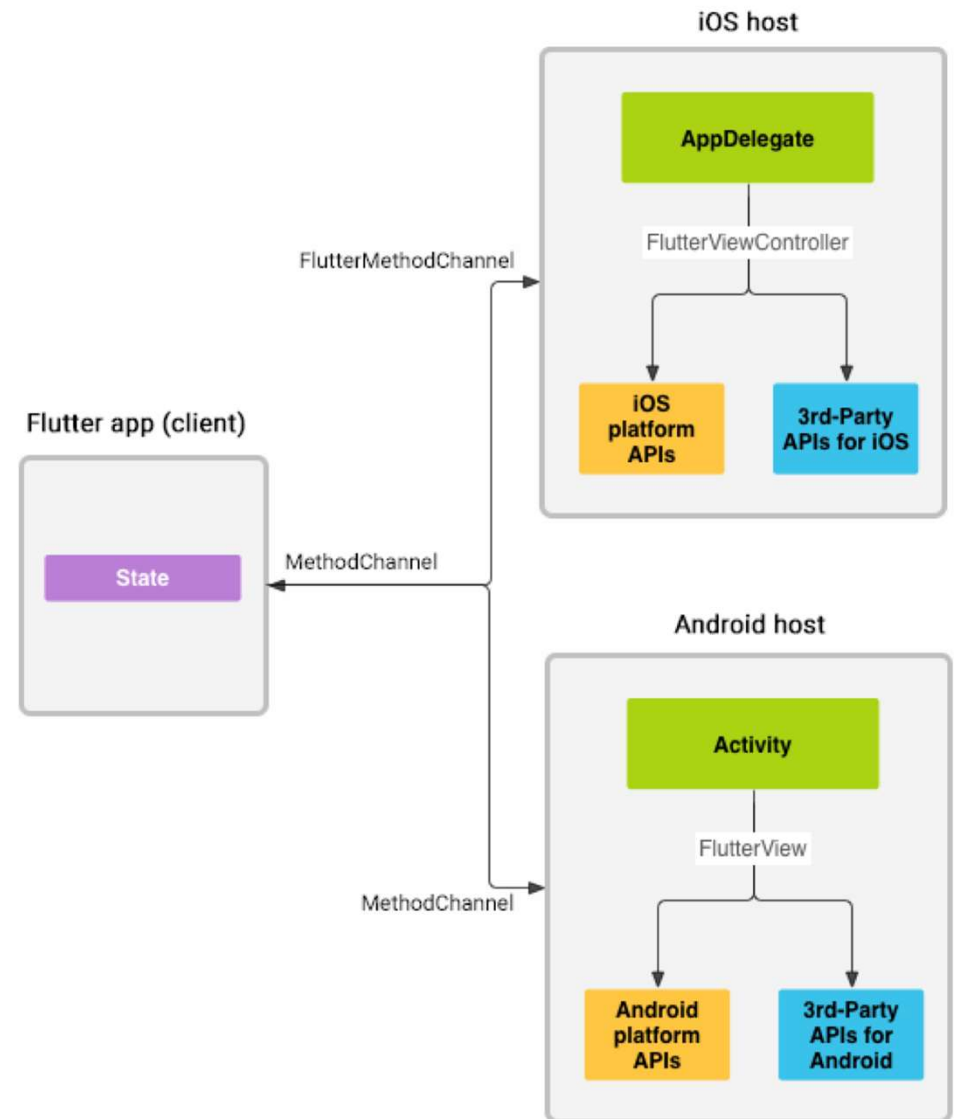
Text editing boxes

- A `TextEditingController` controller can be associated with the editing elements, allowing management of text editing widgets

```
final TextEditingController _tec = TextEditingController();
void initState() {
  super.initState();
  _tec.addListener(() { /* ... */ });
}
void dispose() {
  _tec.dispose();
  super.dispose();
}
...
SizedBox(
  width: 200,
  child: TextFormField(
    decoration: const InputDecoration(
      labelText: 'Increment:',
      hintText: 'Value to increment',
      border: OutlineInputBorder(),
    ),
    controller: _tec,
    //key: Key("$_inc"),
    //initialValue: "$_inc",
    onChanged: (value) => changeInc(int.tryParse(value) ?? 1),
  ),
),
...
```

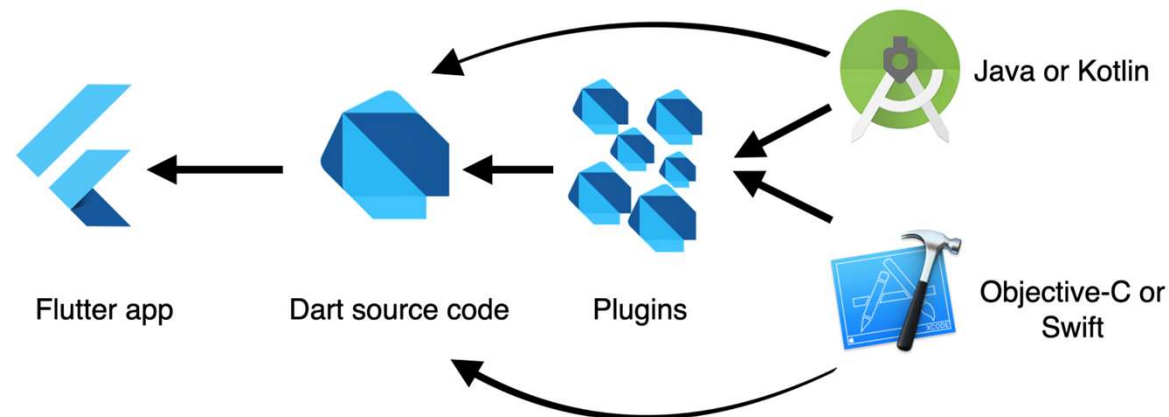
Plugin integration

- In *Flutter* we can add features through *packages* or *plugins*
- **Plugins $\neq$ Packages**,
 - Plugins typically invoke native platform APIs (*Android*, *iOS*, etc.) and usually require native implementation and a bridge (*platform-channel*) to communicate with *Dart*



Plugin integration

- <http://pub.dev> platform must be used to choose the most appropriate *plugin* or *package for each application*
 - It often depends on the habit/history used in each company
 - Added to dependencies in `pubspec.yaml` file
- Dependencies can also be pointed directly to other locations, such as a remote repository



Package integration

- Since *packages* are in *Dart code* (like *package http* used previously) nothing specific to each platform is necessary
 - The programmer does not have to do anything other than consult the documentation and add the dependency

Plugin integration

- In the case of *plugins* ...
 - Sometimes, it is necessary to add specific permissions or settings for each platform
 - For example, for GPS, it is necessary to have permissions specified in the manifest file (*Android*) or `info.plist` (*iOS*)
 - It may also be necessary to add dependencies or restrictions in the configurations of each platform, for example in the `build.gradle` file

Plugin inclusion shared preferences

- Go to <http://pub.dev>
- Search for *plugin* `shared_preferences`
- Check the documentation/version and add it to the application dependencies (`pubspec.yaml`)
- Example of application to store the change to the increment, made in previous *slides*

- Get data

```
Future<void> initInc() async {  
    var prefs = await SharedPreferences.getInstance();  
    setState (() { _inc = prefs.getInt('increment') ?? 1; } );  
}
```

- Write data

```
Future<void> changeInc(int inc) async {  
    setState (() { _inc = inc; } );  
    var prefs = await SharedPreferences.getInstance();  
    await prefs.setInt('increment', _inc);  
}
```


Exercise – localization plugin

- Go to <http://pub.dev>
- Search for *plugin* location
- Check the documentation/version and add it to the application dependencies (`pubspec.yaml`)
- Add specific permissions for *Android* and *iOS* , based on documentation (only mandatory for background location)
 - *Android* : `android/app/src/main/AndroidManifest.xml`

```
<uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
<uses-permission android:name="android.permission.ACCESS_BACKGROUND_LOCATION"/>
```
 - *iOS* : `ios/Runner/Info.plist`

```
<key>NSLocationWhenInUseUsageDescription</key>
<string>We need your location to provide personalized features and services.</string>
```

Exercise

- Change the app to display the user's current location
- Add service state checking and permissions request at *runtime* as described in the documentation
- Display location explicitly via a button

Exercise

```
Location location = new Location();

bool _serviceEnabled=false;
PermissionStatus _permissionGranted = PermissionStatus.denied;
LocationData _locationData = LocationData.fromMap({
  'latitude' : 40.1925,
  'longitude' : -8.4128
});

Future<void> initLocation() async {
  _serviceEnabled = await location.serviceEnabled();
  if (!_serviceEnabled) {
    _serviceEnabled = await location.requestService();
    if (!_serviceEnabled) {
      return;
    }
  }
  _permissionGranted = await location.hasPermission();
  if (_permissionGranted == PermissionStatus.denied) {
    _permissionGranted = await location.requestPermission();
    if (_permissionGranted != PermissionStatus.granted) {
      return;
    }
  }
  getLocation();
}
```

Exercise

```
Future<void> getLocation() async {  
  if (!_serviceEnabled ||  
      _permissionGranted != PermissionStatus.granted) {  
    return;  
  }  
  _locationData = await location.getLocation();  
  setState(() {});  
}
```

```
@override  
Widget build(BuildContext context) {  
  return ...  
  ...  
  Row(  
    mainAxisAlignment: MainAxisAlignment.spaceEvenly,  
    children: [  
      Text('Lat: ${_locationData.latitude}'),  
      Text('Lon: ${_locationData.longitude}'),  
    ],  
  ),  
  ElevatedButton(  
    onPressed: getLocation,  
    child: const Text('Get location')  
  ),  
  ...  
}
```

Exercise

- Automatically update location using *listener* `onLocationChanged`
- Add a *widget* `StreamBuilder<LocationData>` for displaying location data whenever it is updated
- Try changing the location in the simulator settings

Exercise

```
ElevatedButton(  
  onPressed: () {  
    location.onLocationChanged.listen(  
      (LocationData currentLocation) {  
        setState(() {_locationData = currentLocation;});  
      }  
    );  
  },  
  child: const Text('Activate continuous location')  
) ,  
  
StreamBuilder(  
  stream: location.onLocationChanged,  
  builder: (context, location) => Row(  
    mainAxisAlignment: MainAxisAlignment.spaceEvenly,  
    children: [  
      Text('Lat: ${location.data?.latitude}'),  
      Text('Lon: ${location.data?.longitude}'),  
    ],  
  )  
)
```

pick_image and path_provider

```
late Directory dir;
late File file;

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  dir = await getApplicationDocumentsDirectory();
  file = File("${dir.path}/userImage1");

  runApp(const MyApp());
}
...
ElevatedButton(
  onPressed: () async {
    final imgFile = await ImagePicker().pickImage(source: ImageSource.gallery);
    if (imgFile != null) {
      await imgFile.saveTo(file.path);
      FileImage(file).evict(); // avoid cache
    }
    setState(() {});
  },
  child: const Text("Pick Image")
),
if (file.existsSync())
  Image.file(key: UniqueKey(), file),
...
```

Firebase – Firestore

- Create the *Firebase* project
- Add *Firestore* to the project
- Connect the Flutter project to *Firebase*
 - Install *Firebase CLI*:
 - `https://firebase.google.com/docs/cli`
 - Install using *npm*
 - Update the PATH environment variable
 - In the terminal:
 - `firebase login`
 - `dart pub global activate flutterfire_cli`
 - Update the PATH environment variable
 - In the Flutter project folder:
 - `flutterfire configure [--project=<project_ID>]`

Firebase – Firestore

- Add the dependencies:

```
firebase_core: any  
cloud_firestore: any
```

- Initialize *Firebase*:

```
import 'package:firebase_core/firebase_core.dart';  
import 'package:cloud_firestore/cloud_firestore.dart';  
import 'firebase_options.dart';  
  
void initFirebase() async {  
  await Firebase.initializeApp(  
    options: DefaultFirebaseOptions.currentPlatform,  
  );  
}  
  
void main() {  
  // Ensure that Flutter is initialized before Firebase  
  WidgetsFlutterBinding.ensureInitialized();  
  initFirebase();  
  runApp(const MyApp());  
}
```

Firebase – Firestore

- To add a document:

```
String? _error;
```

```
void addFirebase() async {  
  var db = FirebaseFirestore.instance;  
  var document = db.collection('MyColl').doc("MyDocAMOV");  
  await document.set({  
    'subject': 'Mobile Architectures',  
    'nrStudents': 0}  
  ).onError(  
    (error, stackTrace) {  
      setState(() {_error = error.toString();});  
    });  
  
  setState(() {  
    _error = null;  
  });  
}
```

Firebase – Firestore

- To update a document:

```
void updateFirebase() async {  
  var db = FirebaseFirestore.instance;  
  var document = db.collection('MyColl').doc("MyDocAMOV");  
  var data = await document.get(  
    const GetOptions(source: Source.server));  
  if (data.exists) {  
    var nrStudents = data['nrStudents'] + 1;  
    document.update({'nrStudents': nrStudents}).then(  
      (res) => setState(() { _error = null; } ),  
      onError: (e) => setState(() { _error = e.toString(); })  
    );  
  } else {  
    setState(() { _error = "Document doesn't exist"; });  
  }  
}
```

Firebase – Firestore

- To delete a document:

```
void delFirebase() async {  
  var db = FirebaseFirestore.instance;  
  var document = db.collection('MyColl').doc("MyDocAMOV");  
  await document.delete().onError(  
    (error, stackTrace) {  
      setState(() {_error = error.toString();});  
      debugPrint('Error!!!');  
    });  
  
  setState(() {  
    _error = null;  
  });  
}
```

Firebase – Firestore

- The first time, run in the command line with `flutter run` to enable *multidex*
 - Option – modify the `build.gradle` configuration of the module:

```
defaultConfig {  
    ...  
    multiDexEnabled true  
}  
  
dependencies {  
    implementation "androidx.multidex:multidex:2.0.1"  
}
```